WQD7009 BIG DATA APPLICATIONS AND ANALYTICS

CLOUD-BASED ANALYTICS FRAMEWORK FOR DIABETIC PATIENT READMISSION: A DEMOGRAPHIC AND CLINICAL CARE ANALYSIS

GROUP 20

	Name	Matric Number
1.	Faiz Yah Bin Abdul Hanan Yah	24087980
2.	Muhammad Shahir Bin Watimin	17180825
3.	Muhammad Syafiq bin Salim	17107373
4.	Nur Hazirah Binti Kamal Bahrin	24214519
5.	Poonaresi Subramaniam	S2023729

Table of Contents

1.0 Introduction **3**
2.0 Meeting Minutes **4**
3.0 Layers in Data Lifecycle **6**
3.1 Data Ingestion Layer 6
3.2 Data Storage Layer 7
3.3 Data Processing Layer 7
3.4 Data Analytics Layer 8
3.5 Data Visualization and Reporting Layer 8
4.0 Tools Selection and Justification **9**
4.1 Data Ingestion: Google Clouds Storage 9
4.2 Data Processing: Dataproc PySpark 9
4.3 Data Storage: BigQuery 10
4.4 Data Analytics: BigQuery SQL 10
4.5 Data Visualization: Looker Studio 10
5.0 Implementation of the Proposed Framework **11**
5.1 Data Ingestion 11
5.2 Data Processing 11
5.3 Data Storage 16
5.4 Data Analytics 17
5.5 Looker Studio 19
6.0 Evaluation of the Implemented Framework **21**
6.1 Memory Consumption 22
6.2 Data Pipeline Efficiency 23
6.3 Data Throughput 24
7.0 References **25**
8.0 Appendix **26**

1.0 Introduction

The rapid growth in volume, velocity, and variety of data poses challenges for large-scale data management and analysis. With reliance on data for operational decisions, organizations encounter difficulties associated with big data. These include dealing with raw and incomplete data sets, maximizing the use of computational resources in distributed systems, unwieldy data pipelines, and unreliable transport from storage to processing systems, all of which can impede analysis performance and decision-making efficiency.

To achieve this, we propose, implement, and evaluate a cloud-native data engineering framework supporting end-to-end data processing based on managed cloud services that achieves the following objectives: automate high-throughput and fault-tolerant data ingestion and cleaning on a distributed system, build data pipelines easily with integrated cloud-native services, resource-efficiently perform stable and cost-effective data processing, and scale analytical querying and visualization. These objectives address the shortcomings of customary and poorly integrated data engineering workflows.

To validate the proposed framework's feasibility, this study applies the architecture to a real-world healthcare dataset examining diabetic patient readmission patterns. The selected dataset comprises 101,766 records from 130 US hospitals spanning 1998-2008, **capturing comprehensive patient demographics, clinical indicators and readmission outcomes.** This implementation demonstrates the practicality of data-driven insights in healthcare for informing clinical decision-making, while serving as a reference architecture for healthcare organizations and data engineering practitioners addressing similar challenges.

2.0 Meeting Minutes

Date: 6:30pm – 7:04pm, 25th Nov 2025

Attendees: Faiz (Leader), Shahir, Syafiq, Hazirah, Poonaresi

Meeting Objectives

1. To review each member's proposed dataset.
2. To evaluate datasets based on the assignment criteria.
3. To select the most suitable healthcare-related dataset for the project.
4. To delegate tasks for the group assignment.

Comparative Evaluation Matrix

Criteria	Faiz	Shahir	Syafiq	Hazirah	Poona
Proposed Dataset	Diabetes 130-US Hospitals for Years 1999-2008	Diabetes Health Indicators Dataset	Heart Attack Analysis & Prediction Dataset	Liver Disease Patient Dataset 30K train data	Diabetes Health Indicators Dataset
Relevance	High; Diabetes care, readmission and clinical outcomes	High; Risk behavior and predictive analytics	Medium; Individual diagnostic tool	High; Predictive diagnostic tool	High; Risk prediction and population monitoring
Data Quality & Completeness	High; Real clinical data and few missing data.	High; Clean & structured.	Low; Too clean.	High; Complete with a few missing data.	High; Minimal missing values.
Size & Structure	Sufficient; 101k rows	Sufficient; 250k rows	Insufficient; 300 rows	Low volume; 30k rows	Sufficient; 100k rows
Variety & Richness	High variety; 47 features	Medium variety; 22 features	Low variety; 14 features	Low variety; 10 features	High variety; 31 features
Applicability to Healthcare Domain	Uses standard ICD-9 healthcare coding	General risk assessment	Heart attack diagnostic	Clinical research	Metabolic and chronic risk

Meaningful Data Exploration Potential	High; Classification and utilization analysis	High; Risk factor patterns analysis	Limited; Known correlations	High; EDA /Correlation	High; Behavioral analysis
Tool Compatibility	Cloud compatible	Cloud compatible	Low compatibility	Cloud compatible	Cloud compatible
Innovation Opportunities	ML models for readmission	Predictive modelling	IoT Simulation	Predictive modelling	Behavioral analytics
Integration Potential	High; Manageable complexity	High	Low; No standard medical coding	Restricted: Narrow scope	High; Standardize fields

Final Decision & Justification

The group selected the UCI Diabetes 130-US Hospitals dataset through majority voting based on:

- **Scale & Diversity**: 101,766 records with 47 features spanning demographics, clinical data, and medication information
- **Healthcare Relevance**: Addresses critical challenges in hospital utilization and readmission prediction
- **Domain Applicability**: Utilizes standard ICD-9 clinical codes, enabling realistic EHR system integration
- **Technical Complexity**: Real-world data quality issues (missing values, inconsistencies) demonstrate advanced preprocessing capabilities
- **Framework Justification**: Dataset volume and variety warrant robust cloud-based analytics deployment
- **Innovation Potential**: Supports meaningful machine learning exploration for improved clinical outcomes

Task Delegations

Task	Owner(s)
Cloud Framework Architectural Design	Poona
Tool Selection & Justification	Syafiq
Practical Output Evidence Implementation	Faiz & Shahir
Framework Evaluation & Metrics	Hazirah

3.0 Layers in Data Lifecycle

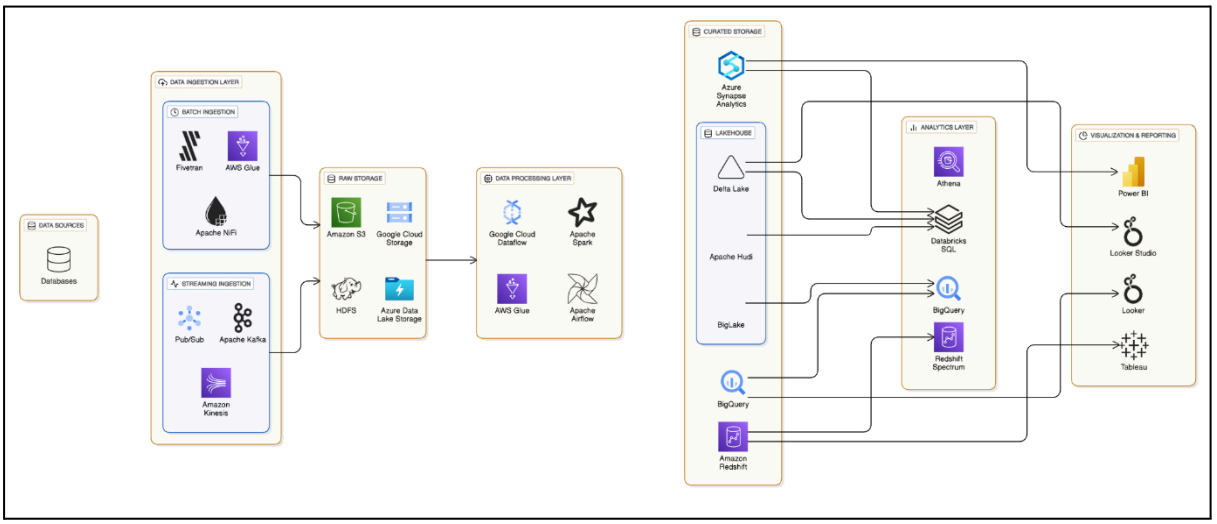


Diagram 1: Typical data lifecycle and layers (Refer to appendix for enhanced version)

The data lifecycle describes the complete end-to-end process and flow of data, from its initial raw state through to its delivery to stakeholders. The lifecycle progresses sequentially through five distinct layers:

i. Data Ingestion
ii. Data Storage
iii. Data Processing
iv. Data Analytics
v. Data Visualization and Reporting Layer

3.1 Data Ingestion Layer

The data ingestion layer is the entry point of a cloud-based data analytics framework. It is responsible for collecting data from multiple source systems and transferring it into the cloud environment for further processing. Its primary role is to ensure that data is ingested in a reliable, secure, and scalable manner so that downstream analytics can operate on complete and timely datasets. Given the diversity of data sources and velocities, the ingestion layer typically supports both batch ingestion and streaming ingestion modes.

Table 1: Comparison between batch ingestion and streaming ingestion

Aspects	Batch Ingestion	Streaming Ingestion
Data Transfer Pattern	Periodic and scheduled	Continuous and real time
Typical use cases	Daily database extracts, historical datasets, periodic reporting	Event-driven systems, real-time monitoring, live analytics
Data Volume	Large volumes ingested in bulk	Continuous flow of small data events
Latency	High latency (minutes to hours)	Low latency (seconds or milliseconds)
Cost Efficiency	More cost-effective for non-time-sensitive data	Higher cost due to real-time infrastructure

3.2 Data Storage Layer

Once data is ingested through batch or streaming pipelines, it is initially stored in cloud object storage as raw data, forming a **data lake** that preserves information in its original format. This raw storage layer serves as a single source of truth, supporting data traceability, auditability, and the ability to reprocess data when analytical or business requirements change. Cloud-native storage solutions such as **Google Cloud Storage, Amazon S3, Azure Data Lake Storage, and HDFS** are commonly used due to their scalability, durability, and cost efficiency.

As data progresses through the framework, selected datasets are cleaned, validated, and transformed before being moved into curated storage. This curated layer is typically implemented as a cloud **data warehouse**, where predefined schemas and data models are enforced to ensure data quality, consistency, and high query performance. Platforms such as **BigQuery, Amazon Redshift, Snowflake, and Azure Synapse Analytics** store analytics-ready data and support reporting, dashboards, and advanced analytics.

3.3 Data Processing Layer

In this layer, processing engines read data from the raw storage layer and apply a series of operations to improve data quality, consistency, and usability before it is made available for downstream analytics and visualization. Typical activities performed in this layer include data cleaning, such as removing duplicate records, handling missing or invalid values, correcting data types, and filtering out erroneous data.

Distributed processing frameworks such as **Apache Spark, Google Cloud Dataflow, AWS Glue, and Apache Flink** are commonly used to execute data processing at scale, while orchestration tools like **Apache Airflow** manage scheduling and workflow dependencies. The outputs of this layer are written to curated storage with enforced schemas, ensuring that analytics, machine learning, and reporting layers operate on reliable and consistent data.

3.4 Data Analytics Layer

The Analytics layer is responsible for analyzing curated, high-quality data to generate insights that support understanding, evaluation, and decision-making. The analytics layer typically supports interactive querying and batch analytical workloads. Analysts use SQL and analytical functions to explore data efficiently, while advanced analytics may include statistical modeling and forecasting. Tools commonly used in this layer include **BigQuery, Presto/Trino, Athena, Databricks SQL, and Redshift Spectrum**. The outputs of the analytics layer are often written back to the data warehouse and consumed by visualization and reporting tools.

3.5 Data Visualization and Reporting Layer

The Visualization and Reporting layer present analytical results in a clear and actionable way for end users. It converts insights from curated data and analytics outputs into charts, dashboards, and reports that support decision-making for both technical and non-technical stakeholders. Common tools used in this layer include **Looker, Looker Studio, Tableau, Power BI, and Superset.** These platforms support interactive dashboards, scheduled reports, and data sharing capabilities. Outputs from this layer may also be embedded into web or mobile applications to support real-time monitoring and decision support.

4.0 Tools Selection and Justification

For this cloud-based analytics framework, we have selected a cohesive Google Cloud Native stack to ensure seamless integration, security and scalability across the data lifecycle. Following is the data architecture diagram integrated in our study:

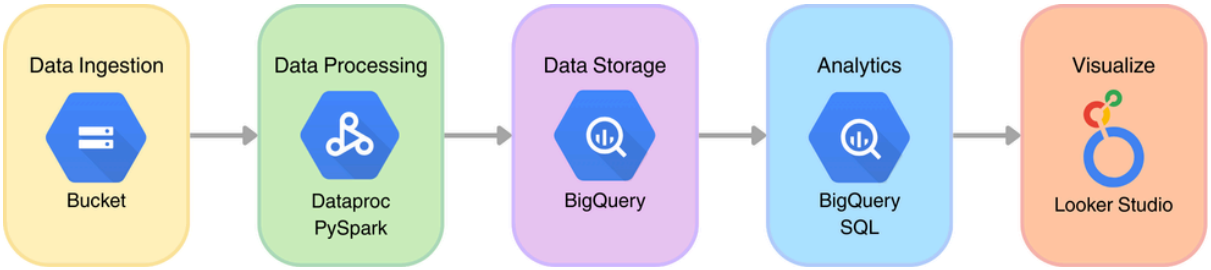


Diagram 2: Proposed Cloud-Based Analytics Framework

4.1 Data Ingestion: Google Clouds Storage

The dataset is a comma-separated values (CSV) file sourced from the UCI Machine Learning Repository making it suitable for ingestion into most pipelines. Google Cloud Storage (GCS) is a scalable, secure and durable object storage service offered by GCP. It organizes data into logical containers that can hold data of any format and any size known as buckets. In modern cloud architecture, GCS typically serves as the Data Lake which acts as the initial repository where raw data is collected and preserved in its original state before any processing or transformation takes place.

4.2 Data Processing: Dataproc PySpark

Dataproc is a fully managed, cloud-native swervice for running Apache Spark and Hadoop clusters, enabling high-speed distributed data processing. In this layer, we utilize Dataproc to execute PySpark jobs leveraging its distributed capabilities to efficiently clean and filter raw CSV data ingested from GCS. Specifically, we will configure a standard cluster with a master node to run a PySpark script that automatically identifies and drops columns with excessive missing values. This strategy ensures only high-quality features are preserved for downstream analysis by automating the elimination of sparse data. Compared to on-premise solutions, Dataproc offers superior agility allowing us to provision fully operational clusters in minutes rather than the weeks required for physical server procurement and manual setup.

4.3 Data Storage: BigQuery

BigQuery is a fully managed, serverless enterprise data warehouse designed for business agility. It utilizes standard SQL, enabling analysts to perform complex transformations without managing infrastructure. In our framework, BigQuery serves as the Central Data Warehouse where raw data is cleaned and prepared for analysis. BigQuery houses the cleaned dataset, utilizing automated schema detection to enforce strict data types such as categorizing demographics as strings and clinical metrics as integers. Additionally, the "Preview" feature enables low-latency verification of data integrity and structure allowing us to validate records instantly without executing costly queries.

4.4 Data Analytics: BigQuery SQL

Apart from storage, we selected BigQuery to leverage its robust Standard SQL for rapid Exploratory Data Analysis (EDA) directly within the console. Its serverless architecture enables instant querying and preliminary visualization of large datasets without external data extraction. We will leverage BigQuery SQL to perform univariate and bivariate analysis directly within the data warehouse. BigQuery’s built-in visualization features also allow us to instantly graph query results, facilitating real-time validation of statistical properties before the data is finalized for the dashboarding layer.

4.5 Data Visualization: Looker Studio

Looker Studio is a fully web-based BI platform that transforms data into interactive reports and dashboards. We intend to utilize this tool to build a unified dashboard that directly visualizes the insights derived from our analysis, effectively meeting the project's requirement to explore both demographic and clinical care angles in a cohesive manner. This centralized interface empowers users with dynamic filters to isolate population segments and correlate clinical decisions with readmission outcomes, transforming statistical findings into actionable, granular insights.

5.0 Implementation of the Proposed Framework

5.1 Data Ingestion

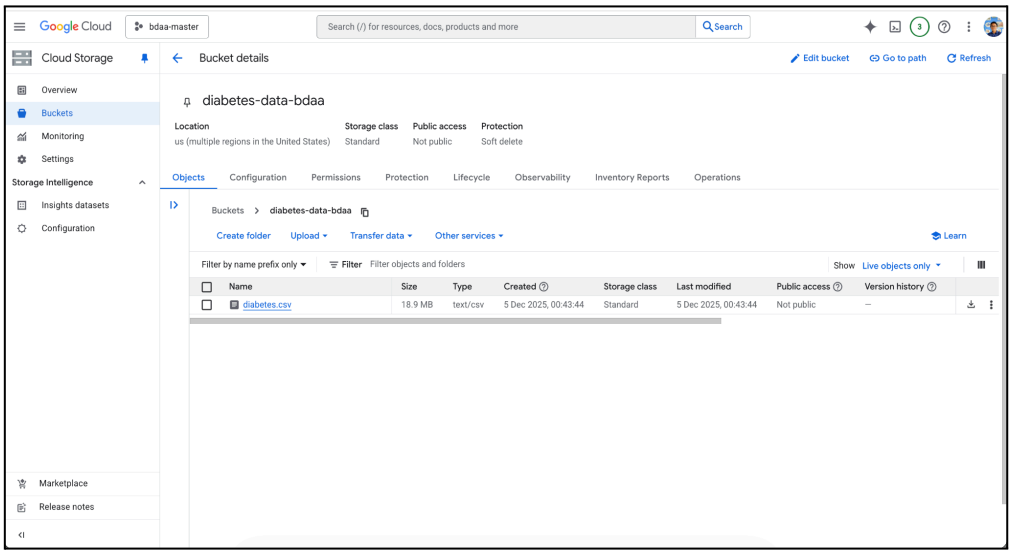


Figure 1: Dataset uploaded to Google Cloud Storage

The data ingestion layer establishes the foundation for the analytics pipeline by collecting and storing raw data in a centralized repository. The UCI diabetes dataset was acquired from Kaggle and uploaded to a Google Cloud Storage (GCS) bucket. As evident in Figure 1, the dataset named *diabetes.csv* was successfully uploaded in a bucket called *diabetes-data-bdaa* and has a file size (18.9 MB) with type (text/csv).

5.2 Data Processing

For data processing, our goal is to drop columns that have a high missing value rate, specifically removing columns with missing percentage of more than 45%. The tools used are Dataproc and PySpark.

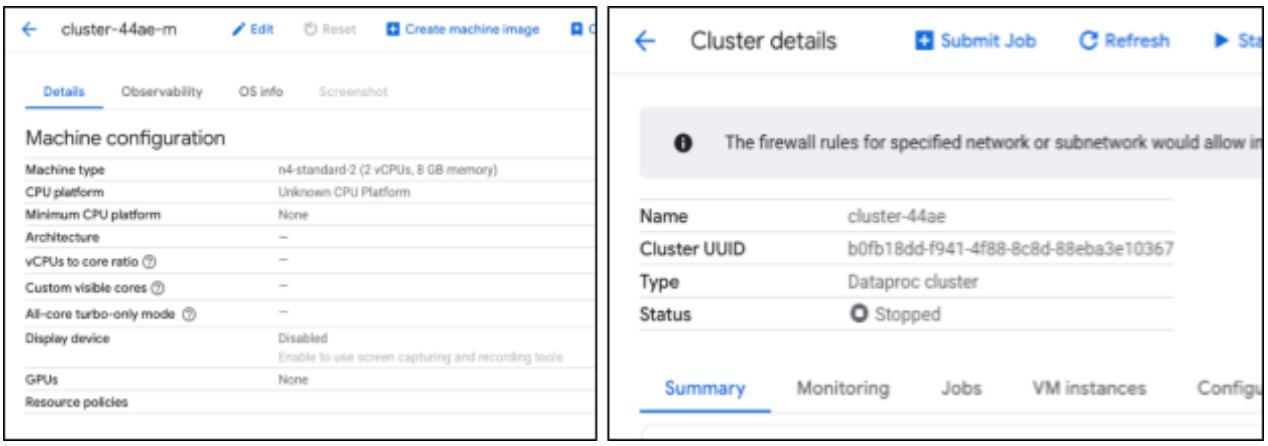


Figure 2: Dataproc cluster and machine configuration

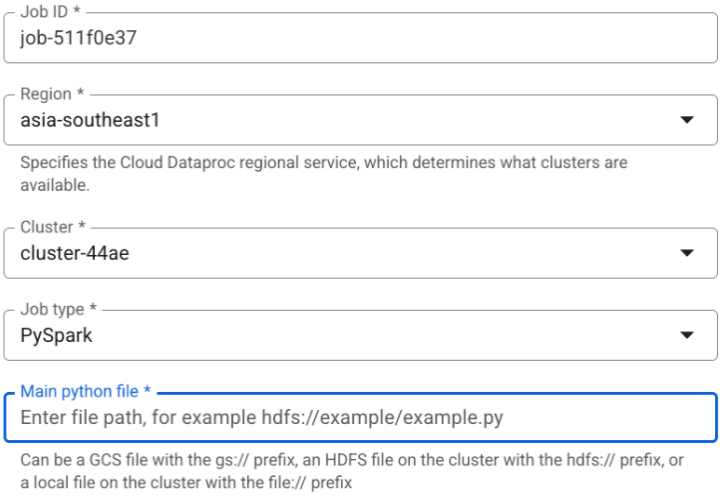


Figure 3: PySpark Job submission configuration

```python
def main():
    # Initialize Spark session
    spark = SparkSession.builder \
        .appName("Hospital-ETL-Cleaning") \
        .getOrCreate()

    # Configuration
    INPUT_PATH = "gs://diabetes-data-bdaa/diabetes.csv"
    OUTPUT_TABLE = "bdaa-master:diabetesbdaa_1.cleaned_data"
    TEMP_BUCKET = "diabetes-data-bdaa"
    THRESHOLD = 0.45

    print("=" * 50)
    print("Starting data cleaning process...")
    print("=" * 50)

    # Read raw data
    print(f"Reading data from {INPUT_PATH}")
    df_raw = spark.read.csv(INPUT_PATH, header=True, inferSchema=True)

    print(f"Raw data shape: {df_raw.count()} rows, {len(df_raw.columns)} columns")

    # Apply cleaning
    df_cleaned = drop_missing_values_spark_optimized(df_raw, threshold=THRESHOLD)

    print(f"Cleaned data shape: {df_cleaned.count()} rows, {len(df_cleaned.columns)} columns")

    # Write to BigQuery
    print(f"Writing to BigQuery: {OUTPUT_TABLE}")
    df_cleaned.write \
        .format("bigquery") \
        .option("table", OUTPUT_TABLE) \
        .option("temporaryGcsBucket", TEMP_BUCKET) \
        .mode("overwrite") \
        .save()

    print("=" * 50)
    print("Data cleaning completed successfully!")
    print("=" * 50)

    spark.stop()
```

```python
def drop_missing_values_spark_optimized(df, threshold=0.45):
    total_rows = df.count()

    agg_exprs = [
        count(when(col(c).isNull() | (col(c) == '?'), c)).alias(c)
        for c in df.columns
    ]

    missing_counts = df.agg(*agg_exprs).collect()[0].asDict()

    columns_to_keep = [
        col_name for col_name in df.columns
        if missing_counts[col_name] / total_rows <= threshold
    ]

    return df.select(columns_to_keep)

if __name__ == "__main__":
    main()
```

Figure 4: PySpark script to drop columns with high missing values (Refer to appendix for enhanced version)

Firstly, a Dataproc cluster with one master node was provisioned to execute a PySpark job (Figure 2). Then a job leveraging PySpark is established where the location to the PySpark script is also defined (Figure 3). The PySpark script (Figure 4) calculates the missing value rate for each column and drops the columns that have missing percentage higher than 45%.

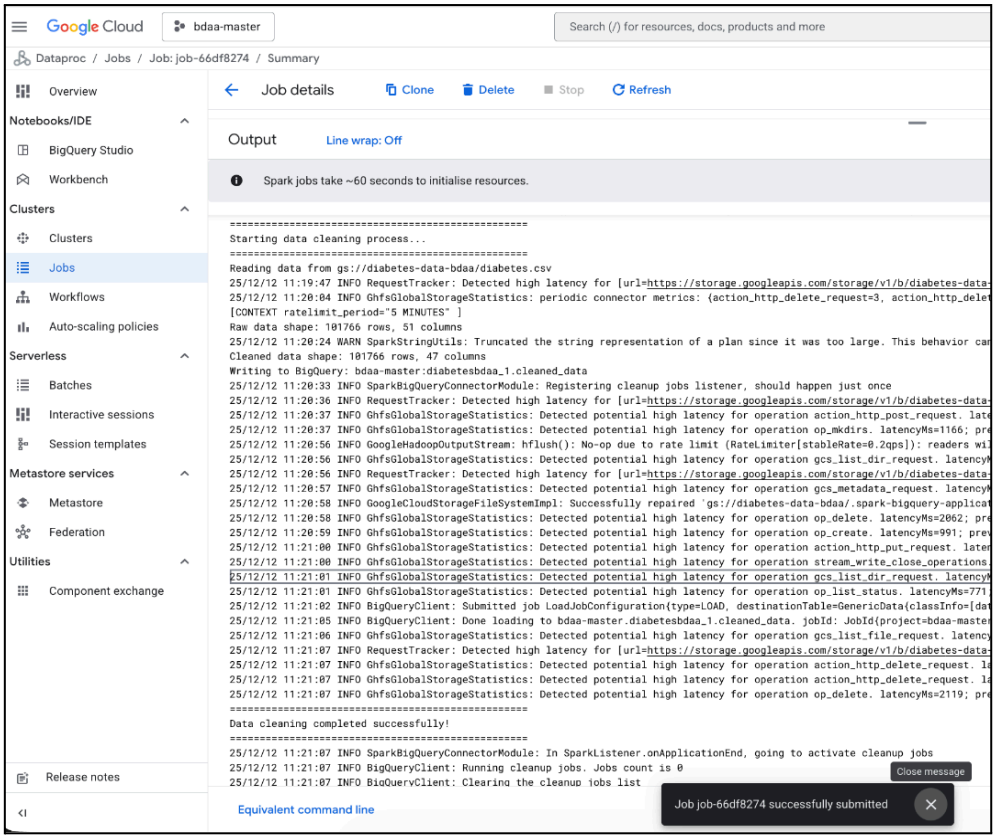


Figure 5: Job log of successful PySpark Job using Dataproc

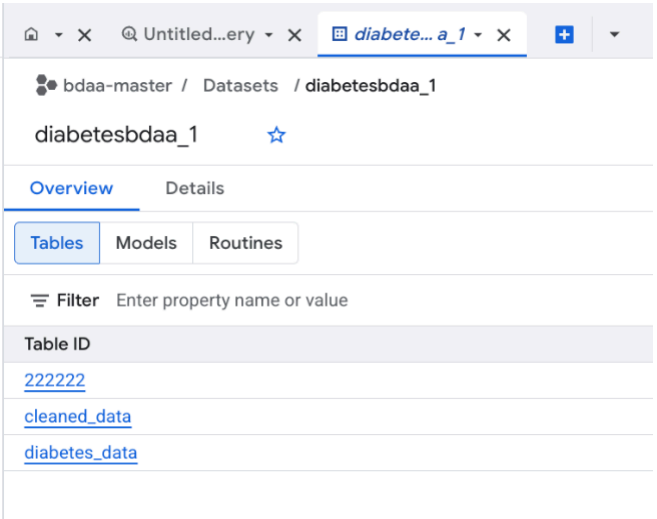


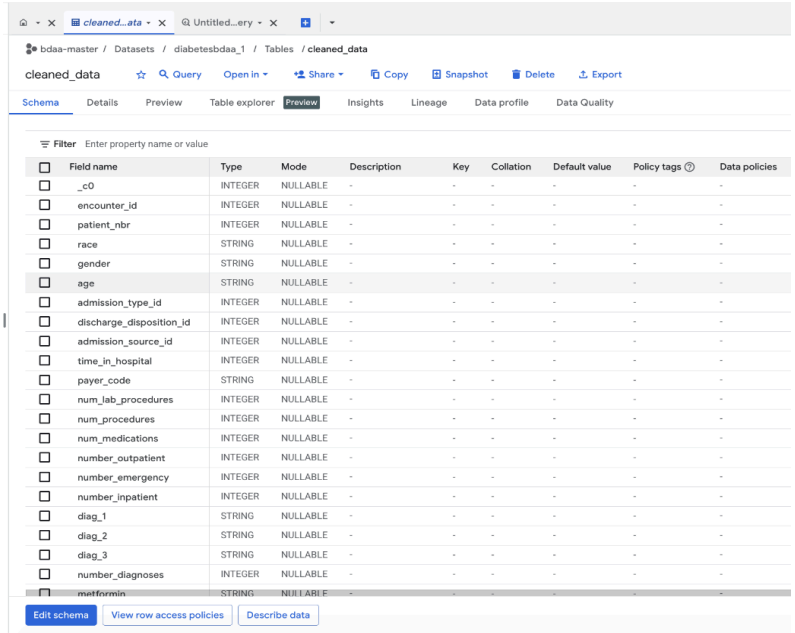
Figure 6: Cleaned data successfully loaded into BigQuery

From the log of the job, it indicates that the job has been successfully executed (Figure 5). The original shape of the data is 101,766 rows with 51 columns, after PySpark data cleaning operation, the cleaned data shape is 101,766 rows with 47 columns, indicating that 5 columns with high missing value rate have been removed. Then, we proceed to check the cleaned

dataset in the BigQuery table (Figure 6). The dataset is present, indicating the success of our job.

5.3 Data Storage

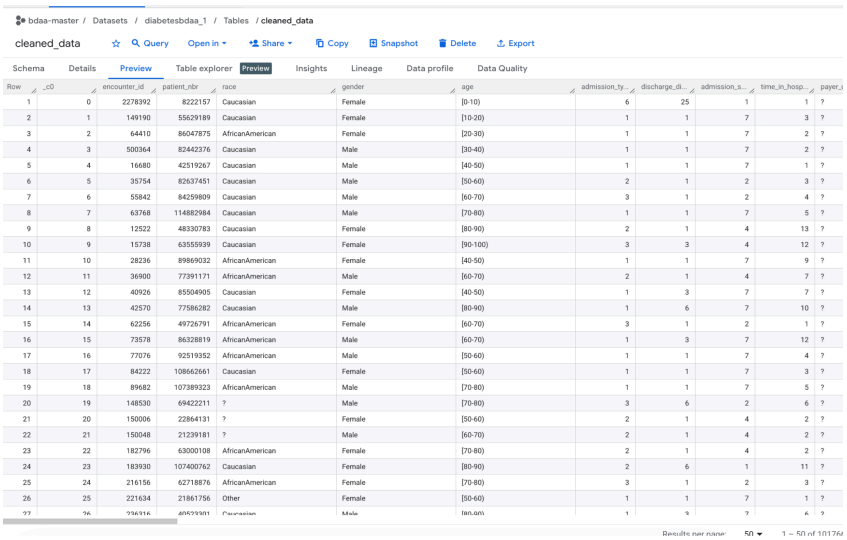
Big Query serves as a data warehouse that stores structured data that has been processed and cleaned. In our case, the cleaned dataset is called cleaned_data.



Field name	Type	Mode	Description	Key	Collation	Default value	Policy tags	Data policies
_c0	INTEGER	NULLABLE	-	-	-	-	-	-
encounter_id	INTEGER	NULLABLE	-	-	-	-	-	-
patient_nbr	INTEGER	NULLABLE	-	-	-	-	-	-
race	STRING	NULLABLE	-	-	-	-	-	-
gender	STRING	NULLABLE	-	-	-	-	-	-
age	STRING	NULLABLE	-	-	-	-	-	-
admission_type_id	INTEGER	NULLABLE	-	-	-	-	-	-
discharge_disposition_id	INTEGER	NULLABLE	-	-	-	-	-	-
admission_source_id	INTEGER	NULLABLE	-	-	-	-	-	-
time_in_hospital	INTEGER	NULLABLE	-	-	-	-	-	-
payer_code	STRING	NULLABLE	-	-	-	-	-	-
num_lab_procedures	INTEGER	NULLABLE	-	-	-	-	-	-
num_procedures	INTEGER	NULLABLE	-	-	-	-	-	-
num_medications	INTEGER	NULLABLE	-	-	-	-	-	-
number_outpatient	INTEGER	NULLABLE	-	-	-	-	-	-
number_emergency	INTEGER	NULLABLE	-	-	-	-	-	-
number_inpatient	INTEGER	NULLABLE	-	-	-	-	-	-
diag_1	STRING	NULLABLE	-	-	-	-	-	-
diag_2	STRING	NULLABLE	-	-	-	-	-	-
diag_3	STRING	NULLABLE	-	-	-	-	-	-
number_diagnoses	INTEGER	NULLABLE	-	-	-	-	-	-
metformin	STRING	NULLABLE	-	-	-	-	-	-

Figure 7: Dataset in Big Query data warehouse

The table schema describes the data field name with their respective data type. Taking a few examples from the (Figure7), *race*, *gender* and *age* are columns with string data type while *num_procedures* and *num_medications* have the integer type instead.



Row	_c0	encounter_id	patient_nbr	race	gender	age	admission_ty...	discharge_di...	admission_s...	time_in_hosp...	payer_c
1	0	2278392	8222157	Caucasian	Female	[0-10)	6	25	1	1	?
2	1	149190	55629189	Caucasian	Female	[10-20)	1	1	7	3	?
3	2	64410	86047875	AfricanAmerican	Female	[20-30)	1	1	7	2	?
4	3	500364	82442376	Caucasian	Male	[30-40)	1	1	7	2	?
5	4	16680	42519267	Caucasian	Male	[40-50)	1	1	7	1	?
6	5	35754	82637451	Caucasian	Male	[50-60)	2	1	2	3	?
7	6	55842	84259809	Caucasian	Male	[60-70)	3	1	2	4	?
8	7	63768	114882984	Caucasian	Male	[70-80)	1	1	7	5	?
9	8	12522	48330783	Caucasian	Female	[80-90)	2	1	4	13	?
10	9	15738	63555939	Caucasian	Female	[90-100)	3	3	4	12	?
11	10	28236	89869032	AfricanAmerican	Female	[40-50)	1	1	7	9	?
12	11	36900	77391171	AfricanAmerican	Male	[60-70)	2	1	4	7	?
13	12	40926	85504905	Caucasian	Female	[40-50)	1	3	7	7	?
14	13	42570	77586282	Caucasian	Male	[80-90)	1	6	7	10	?
15	14	62256	49726791	AfricanAmerican	Female	[60-70)	3	1	2	1	?
16	15	73578	86328819	AfricanAmerican	Male	[60-70)	1	3	7	12	?
17	16	77076	92519352	AfricanAmerican	Male	[50-60)	1	1	7	4	?
18	17	84222	108662661	Caucasian	Female	[50-60)	1	1	7	3	?
19	18	89682	107389323	AfricanAmerican	Male	[70-80)	1	1	7	5	?
20	19	148530	69422211	?	Male	[70-80)	3	6	2	6	?
21	20	150006	22864131	?	Female	[50-60)	2	1	4	2	?
22	21	150048	21239181	?	Male	[60-70)	2	1	4	2	?
23	22	182796	63000108	AfricanAmerican	Female	[70-80)	2	1	4	2	?
24	23	183930	107400762	Caucasian	Female	[80-90)	2	6	1	11	?
25	24	216156	62718876	AfricanAmerican	Female	[70-80)	3	1	2	3	?
26	25	221634	21861756	Other	Female	[50-60)	1	1	7	1	?

Figure 8: Preview cleaned dataset in BigQuery

The preview feature showcases the examples of the actual data. From this, we can gain a quick view and understanding of the data without querying it.

5.4 Data Analytics

Query results

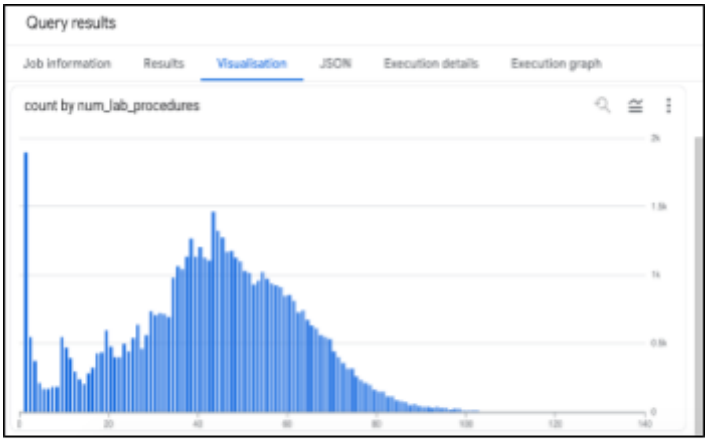
Job information | Results | Visualisation | JSON

Row	readmitted	count
1	NO	54864
2	>30	35545
3	<30	11357

```
SELECT readmitted, COUNT(readmitted) AS count
FROM `bdaa-master.diabetesbdaa_1.cleaned_data`
GROUP BY readmitted
```

Figure 9: BigQuery SQL query for target variable distribution

From a univariate analysis standpoint, we can identify frequency of unique values for categorical columns. For example, we can query to identify the distribution of the target variable, with the result showing an imbalance dataset (Figure 9).

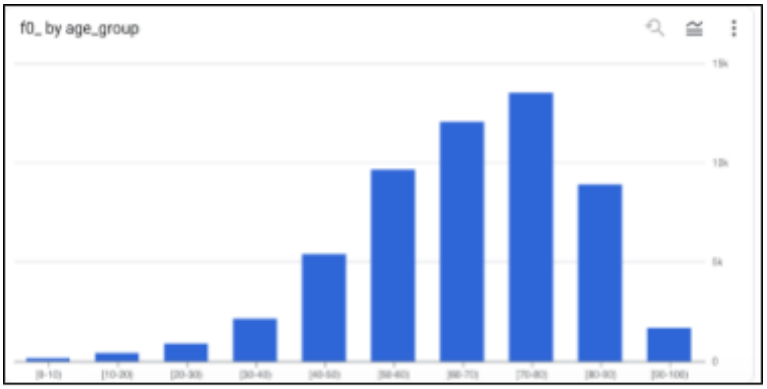



```
SELECT num_lab_procedures, readmitted, COUNT(readmitted) AS count
FROM `bdaa-master`.`diabetesbdaa_1`.`cleaned_data`
GROUP BY num_lab_procedures, readmitted
```

Figure 10: BigQuery SQL query for distribution of laboratory procedures

Next, we can query to identify the distribution of numerical columns, where we will use *num_lab_procedure* column as an example. Once queried, we can leverage on the

‘Visualisation’ tab to seamlessly understand the distribution, with the example resembling a Gaussian distribution (Figure 10).




```
SELECT age AS age_group, readmitted, COUNT(readmitted)
FROM `bdaa-master.diabetesbdaa_1.cleaned_data`
GROUP BY age, readmitted
```

Figure 11: BigQuery SQL query for age group distribution

Showcasing another example, using the *age* column and the proper query, we can obtain that distribution of the age group, providing us insights that people of the older age group are the majority in the sample (Figure 11).

Row	gender_group	readmitted	f0_
1	Female	NO	29038
2	Female	>30	19518
3	Male	NO	25823
4	Male	>30	16027
5	Male	<30	5205
6	Female	<30	6152
7	Unknown/Invalid	NO	3

```
SELECT gender AS gender_group, readmitted, COUNT(readmitted)
FROM `bdaa-master.diabetesbdaa_1.cleaned_data`
GROUP BY gender, readmitted
```

Figure 12:BigQuery SQL query for bivariate analysis of gender and readmission

Extending from there, bivariate analysis can be carried out where aggregate different columns to understand the relationship between columns. Taking an example, we can identify the relationship between gender and readmission rate. Figure 12 shows the breakdown of each gender paired by each readmission category, providing a granular understanding.

5.5 Looker Studio

Looker Studio is a BI dashboard which allows us to encapsulate the different data visualizations in one centralized place.

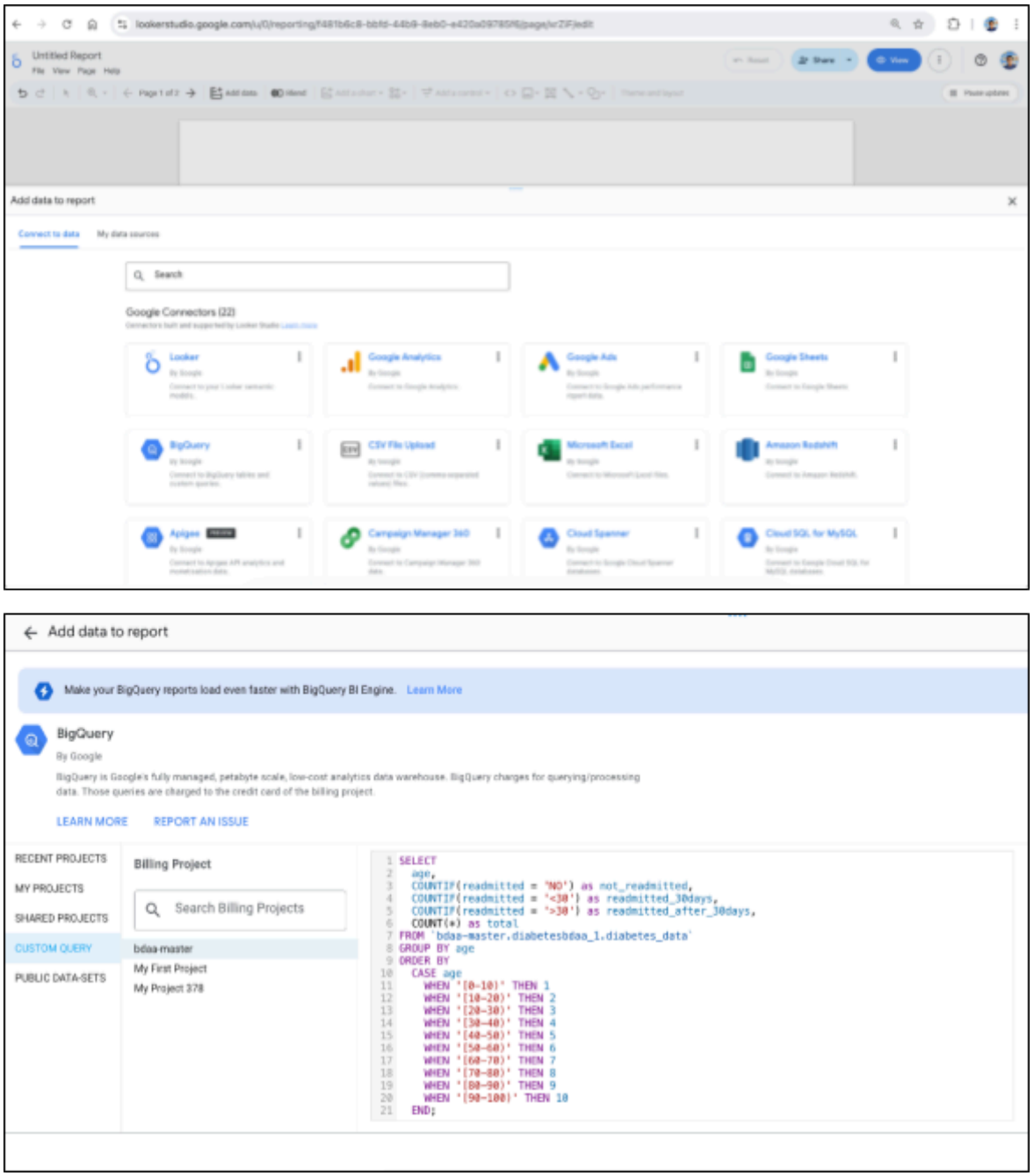


Figure 13: Looker Studio data source configuration using BigQuery

The data required can be directly and conveniently imported from BigQuery. If needed, we can also carry out custom BigQuery SQL queries to obtain more complex data for our dashboard (Figure 13).

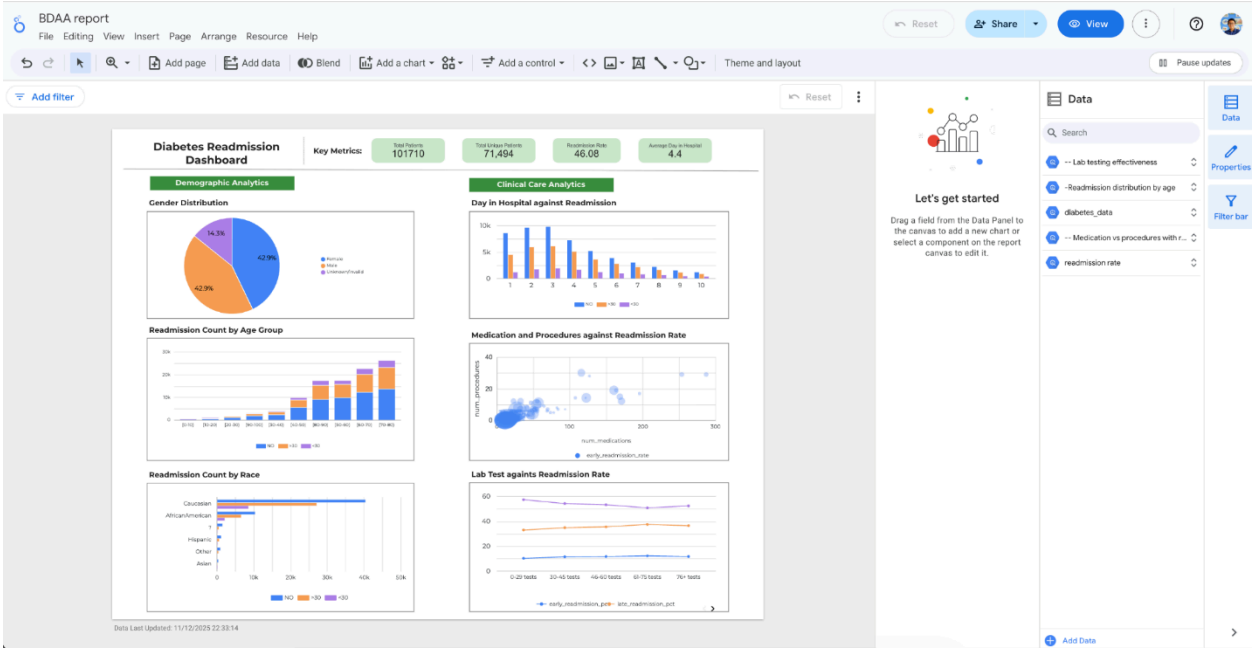


Figure 14: Comprehensive BI Dashboard on Looker Studio (Refer to appendix for enhanced version)

A neat, informative and impactful Looker Studio BI dashboard can be developed efficiently without much hassle. Varying types of data visualizations and data aggregation were deployed for different types of insights, with the purpose of effectively conveying a data-driven narrative. From the dashboard in Figure 14, stakeholders can easily and accurately obtain the needed information in real-time.

6.0 Evaluation of the Implemented Framework

In assessing the performance characteristics of the proposed data engineering framework, we evaluated three performance metrics: **memory consumption**, **data pipeline efficiency**, and **data throughput**. These metrics were selected to critically assess how effectively the framework manages computational resources, orchestrates end-to-end data processing, and sustains reliable data movement under realistic execution conditions.

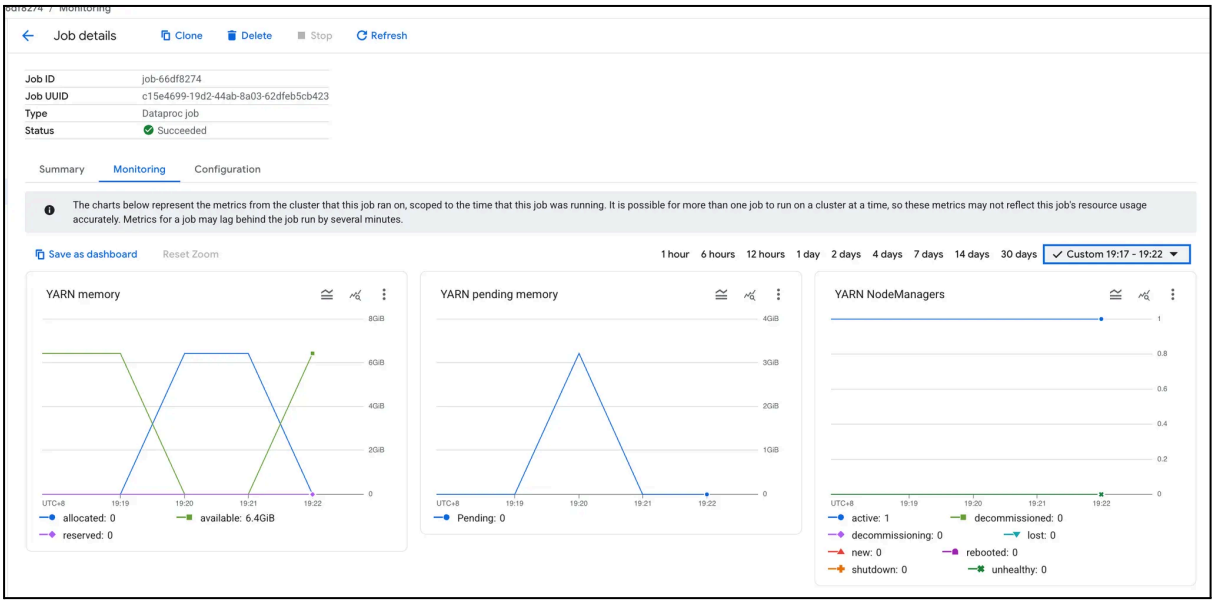


Figure 15: Dataproc job monitoring metrics (part 1 of 2)

Figure 16: Dataproc job monitoring metrics (part 2 of 2)

6.1 Memory Consumption

Memory consumption is analysed to identify how efficiently the proposed framework uses **Dataproc** YARN monitoring metrics that manages cluster memory resources during distributed data processing. This is a good metric because it provides utilization of memory resources of the cluster when the job is running. According to the YARN memory dashboard, there was 6.4 GiB memory available in the cluster. No memory was allocated or held long-term. Peak YARN pending memory was around 3 GiB, which was followed by a drop to zero, indicating that this is just a scheduler request and not contention. No job execution latency was observed because NodeManager had a steady state of 1 active with any loss of nodes.

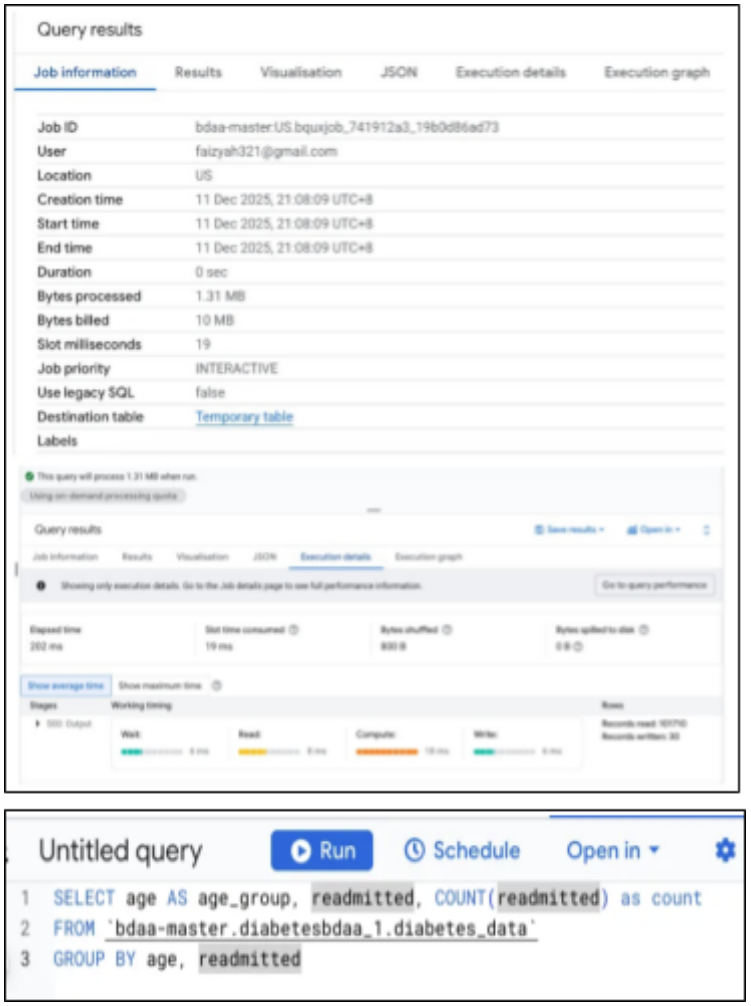


Figure 17: BigQuery SQL job execution details

As for **BigQuery SQL**, looking at Figure 17, the query utilised 1.31MB in only a matter of 18ms to compute the whole process. Albeit the dataset is relatively modest in size,

this remarkable performance further demonstrates BigQuery's efficiency as a data querying tool with minimal execution time and optimised resources utilisation.

6.2 Data Pipeline Efficiency

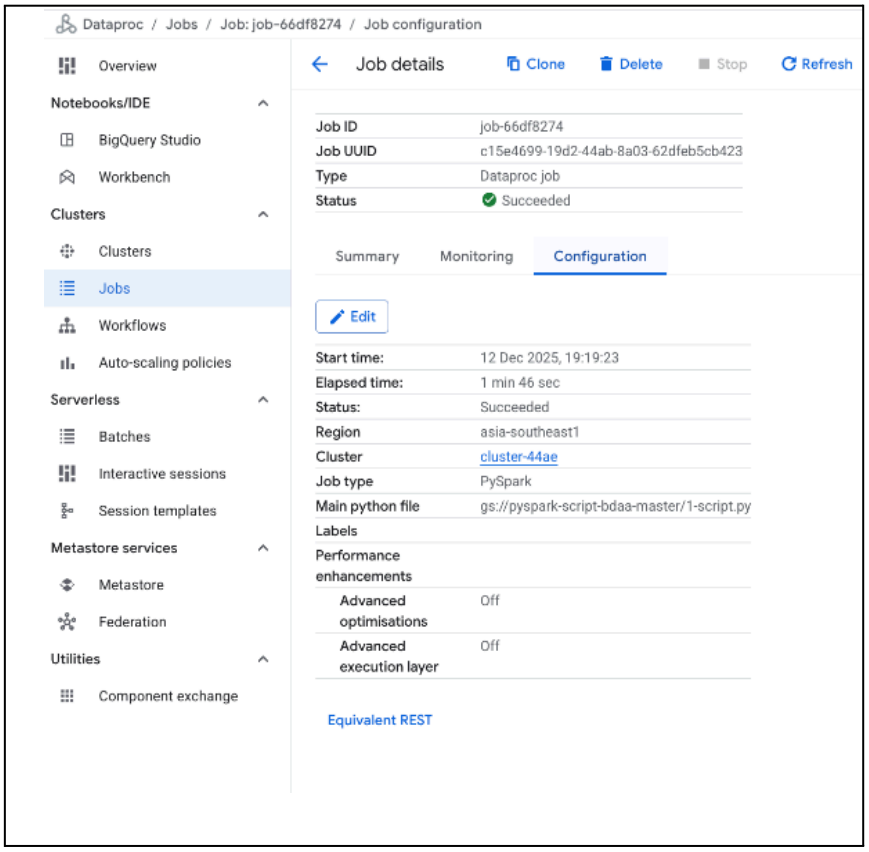


Figure 18: PySpark job configuration and runtime statistics

To evaluate the performance of the whole data pipeline, we observed the pipeline run-time characteristics. As illustrated in Figure 18 , the **Dataproc** job ran successfully in 1 minute 46 seconds. This indicates that all the distributed processing stages were scheduled successfully. According to Figure 15 and Figure 16, CPU utilization averaged between 20% and 60% during runtime, indicating balanced workload distribution across cluster nodes without bottlenecks.

Network and disk I/O patterns further confirm pipeline efficiency with disk operations maintained at moderate levels with read operations at 50.78/s and write operations at 13.27/s. The absence of significant disk spilling indicates that intermediate processing results were retained in memory, thereby minimizing disk I/O overhead and reducing overall latency.

6.3 Data Throughput

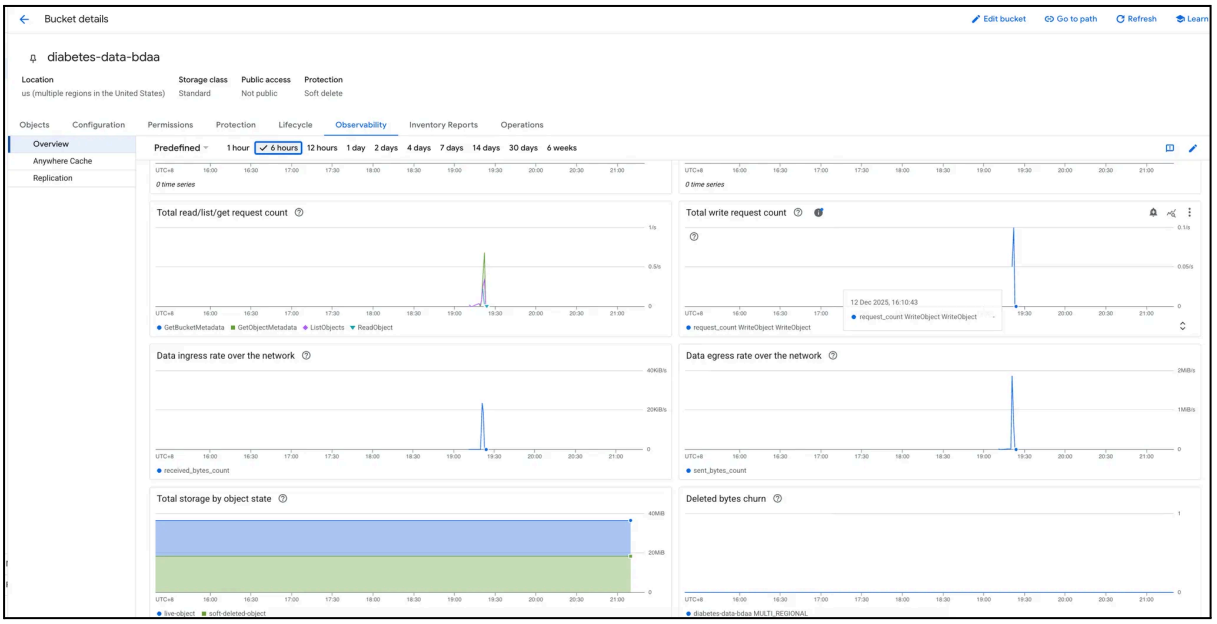


Figure 19: Google Cloud Storage ingestion throughput metrics

Data throughput was analysed to evaluate transfer efficiency between **Google Cloud Storage** and downstream processing components. As seen above, observability metrics reveal that read/list operations peaked at approximately 0.7/s requests during data ingestion, while write operations spiked at 0.1/s requests during output staging.

Network transfer patterns show data ingress peaking at over 20 KiB/s during initial loading, with egress reaching approximately 2 MiB/s during result writing. These concentrated transfer bursts occurring within brief time windows rather than sustained periods indicate efficient batch-oriented data movement where files are transferred in consolidated operations. This throughput pattern aligns well with the framework's batch processing architecture, ensuring data availability when needed while minimizing unnecessary I/O overhead and storage access costs, confirming adequate capacity without introducing pipeline bottlenecks.

Wordcount: 3140 (excluding title page, table of contents and references)

7.0 References

SentinelOne. (2025, September 23). *What is data ingestion? Types, challenges and best practices*. Retrieved from https://www.sentinelone.com/cybersecurity-101/data-and-ai/data-ingestion/ SentinelOne

OvalEdge. (2025, December 1). *6 best data ingestion tools for scalable data pipelines*. Retrieved from https://www.ovaledge.com/blog/data-ingestion-tools ovaledge.com

Domo. (n.d.). *10 best data ingestion platforms in 2025 for real-time analytics and AI*. Retrieved from https://www.domo.com/learn/article/data-ingestion-platforms#:~:text=1.,organizations%20looking%20to%20democratize%20data

Monte Carlo Data. (2024, June 15). *What is a data platform and how to build one?*. Retrieved from https://www.montecarlodata.com/blog-what-is-a-data-platform-and-how-to-build-one/

8.0 Appendix

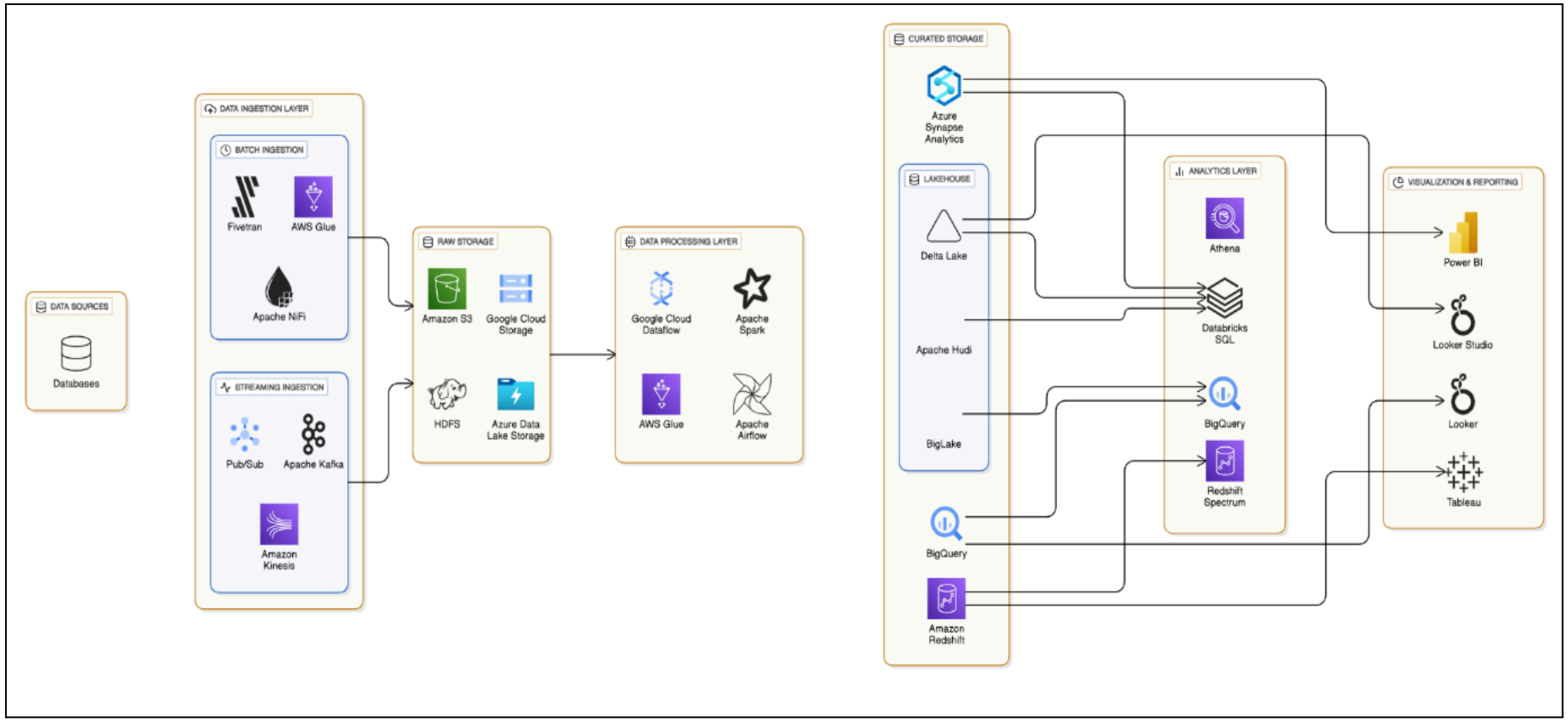


Diagram 1: Typical data lifecycle and layers

```python
def drop_missing_values_spark_optimized(df, threshold=0.45):
    total_rows = df.count()

    agg_exprs = [
        count(when(col(c).isNull() | (col(c) == '?'), c)).alias(c)
        for c in df.columns
    ]

    missing_counts = df.agg(*agg_exprs).collect()[0].asDict()

    columns_to_keep = [
        col_name for col_name in df.columns
        if missing_counts[col_name] / total_rows <= threshold
    ]

    return df.select(columns_to_keep)

if __name__ == "__main__":
    main()
```

```python
def main():
    # Initialize Spark session
    spark = SparkSession.builder \
        .appName("Hospital-ETL-Cleaning") \
        .getOrCreate()

    # Configuration
    INPUT_PATH = "gs://diabetes-data-bdaa/diabetes.csv"
    OUTPUT_TABLE = "bdaa-master:diabetesbdaa_1.cleaned_data"
    TEMP_BUCKET = "diabetes-data-bdaa"
    THRESHOLD = 0.45

    print("=" * 50)
    print("Starting data cleaning process...")
    print("=" * 50)

    # Read raw data
    print(f"Reading data from {INPUT_PATH}")
    df_raw = spark.read.csv(INPUT_PATH, header=True, inferSchema=True)

    print(f"Raw data shape: {df_raw.count()} rows, {len(df_raw.columns)} columns")

    # Apply cleaning
    df_cleaned = drop_missing_values_spark_optimized(df_raw, threshold=THRESHOLD)

    print(f"Cleaned data shape: {df_cleaned.count()} rows, {len(df_cleaned.columns)} columns")

    # Write to BigQuery
    print(f"Writing to BigQuery: {OUTPUT_TABLE}")
    df_cleaned.write \
        .format("bigquery") \
        .option("table", OUTPUT_TABLE) \
        .option("temporaryGcsBucket", TEMP_BUCKET) \
        .mode("overwrite") \
        .save()

    print("=" * 50)
    print("Data cleaning completed successfully!")
    print("=" * 50)

    spark.stop()
```

Figure 4: PySpark script to drop columns with high missing values

Figure 14: Comprehensive BI Dashboard on Looker Studio